Hands-On(ish) LLMOps

Maybe Build · Maybe Demo of an Observable AI System in 90 Minutes

Jeanne McClure, PhD
AI: Principal · Consultant · Expert
Ars Innovate Technologies & Consulting

Postdoctoral Fellow
Data Science & AI Academy
North Carolina State University

DevOps Days · 2026

This material is based in part upon work supported by the National Science Foundation under Grant #DGE-2222148. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

© 2026 Jeanne McClure, PhD / Arsinnovate. Non-commercial use only. Attribution required.

You shipped the LLM feature. It works — mostly.

Last week

your AI assistant gave a customer bizarre advice. Nobody can reproduce it. Nobody knows why.

The logs show an HTTP 200.

Helpful.

You cannot answer:

- **Can you debug it when it breaks?**
- **Do you know what it costs per request?**
- **Can you trace why it gave a wrong answer?**

Today fits inside a bigger practice.

Phase 1
Human Evaluation

Humans score model output by hand against a written rubric. This is how you build ground truth before you decide which model to use.

Phase 2
LLM-as-Judge

Your Phase 1 scores become training data for an automated judge. The judge applies your rubric at scale, across thousands of queries.

Phase 3 - Today
Production Monitoring

Continuous observability of the live system. Traces, costs, drift. This is what we build today. The next 90 minutes lives here.

LLMs fail differently than traditional software.

Traditional Service

- Breaks with a stack trace
- APM sees the failure
- Deterministic: same input, same output
- HTTP 500 means something is wrong

LLM-Powered Service

- Fails with plausible wrong answers
- APM sees a 200 OK
- Stochastic: same input, different output
- HTTP 200 means nothing about quality

Three signals unique to LLM systems.

Cost

Every call has variable cost. Token counts per endpoint, per user cohort, per feature flag are the only honest cost model.

Quality

Prompt + completion content is the only way to know WHY the model answered the way it did. Without it, debugging is guessing.

Grounding

For RAG: the retrieved chunks ARE the ground truth.
Wrong chunks in = "hallucinations" out.
You must log retrieval.

A RAG system. Fully observable. On your laptop.

Load	→	Chunk	→	Embed	→	Retrieve	→	Generate
3 docs		~20 pieces		all-minilm		top-k		llama3.2:1b

+ Langfuse traces · + OpenTelemetry spans · + token, latency, cost

Everything local. Every line yours. No API keys, no cloud, no vendor lock-in.

Four tools. Each does one thing well.

Ollama

Runs the LLM + embeddings locally

llama3.2:1b (~1.3GB) · all-minilm (~46MB)

ChromaDB

Vector store

In-process, no daemon, no config

Langfuse

LLM-native observability

Self-hosted, OSS, understands prompts + tokens

OpenTelemetry + Jaeger

Cross-service tracing · clickable trace UI

Today: Jaeger UI at localhost:16686 · tomorrow: any APM

Five steps. Checkpoints. No one left behind.

0:00	**Intro · verify prework** *Short. If your check_env.py is red, raise a hand now.*
0:08	**Step 1 — minimal RAG** *Works. Black box. No observability.*
0:23	**Step 2 — Langfuse** *Every call traced. Prompts + tokens visible.*
0:43	**Step 3 — OpenTelemetry + Jaeger** *Same pattern, spans you can click on at localhost:16686.*
0:58	**Step 4 — break + debug** *Wrong answers. You find the bug in traces.*
1:18	**Production patterns + Q&A** *What to do at work Monday.*

How to survive a 90-minute live build.

Checkpoints save you

Fall behind?
Type: git checkout step-N — clean working tree, no shame.
The goal is learning, not typing.

Pair up — on purpose at Steps 1 + 4

Steps 1 + 4 are designed for pair/group discussion. Find a neighbor now — demo-watchers and laptop-runners both contribute.
If your setup didn't finish, you're a demo-watcher today. Your packet keeps you in the game.

Stop me when you're lost

I'd rather lose a minute than lose a concept. Raise a hand.

The bug in step 4 is the point

Don't peek at SOLUTION.md. The debugging IS the workshop.

Get the repo. Now — while I'm talking.

Type one URL

bit.ly/3ONJiqb
Redirects to github.com/gigimcc4/DevOpsDays_llmops. Bookmark it now — you'll come back tonight.
No git? Same URL, click green Code button, Download ZIP.

Or clone in your terminal

git clone https://github.com/gigimcc4/DevOpsDays_llmops.git
Then: cd DevOpsDays_llmops

Open it in your IDE

VS Code, Positron (Jeanne's pick today), Cursor, PyCharm — whatever you have. You'll want it open to read each step's rag.py.

Company laptop locked down?

Get the URL anyway. Run it on a personal machine tonight. You leave with the repo either way.

One command. Six things installed.

One command, two minutes (after Docker is up)

Run it
chmod +x setup.sh && ./setup.sh
Mac/Linux/WSL only. Re-run anytime — idempotent.

What setup.sh installs

Ollama + 1.5 GB of models · Python packages · Langfuse (Docker) · Jaeger (Docker)
First run: 5–8 min. Second run: seconds.

What it WON'T install

Python 3.10+ and Docker Desktop. Don't have those? You're a demo-watcher today.

Success looks like

Langfuse at localhost:3000 · Jaeger at localhost:16686 · check_env.py all green.

Hiccups happen. Demo mode is fine.

We can't install Python or Docker for you in 90 minutes

Conference wifi
1.5 GB of model weights on shared wifi = slow.
Watch the demo. Run it tonight at home.

Corp laptop locked down

Blocked Docker, blocked pip, blocked curl|sh?
Watch the demo. Run it on a personal machine.

No Docker Desktop installed

setup.sh checks first and exits clean. Watch the demo. Install at home.

Windows without WSL

setup.sh is Mac/Linux/WSL. WSL is a 5-minute install — do it tonight.

Three short docs. That's our knowledge base.

What our RAG retrieves from

devops_glossary.md
Defines trace, span, OpenTelemetry, Langfuse, vector, ChromaDB, top-k, Ollama.
The vocabulary.

llm_observability.md

Why APMs miss LLM failures.
The three signals: cost, quality, grounding.

rag_patterns.md

Five RAG stages and how each fails. The work-backwards-through-the-trace debug flow.

All three are in your packet

Print copies. Refer back when you ask: did the model use the right source?

STEP 1

Minimal RAG

80 lines. Load → chunk → embed → retrieve → generate. Works. Also, a black box.

```
python step1_rag/rag.py
```

15 min · talk through it, type together, run it, discuss the answers.

STEP 1 · WHAT TO NOTICE

It works. Now ask: what can't you answer?

rag.py · the five stages

```
load_documents()
  read *.md from sample_docs/
chunk_documents()
  500-char chunks, 50 overlap
embed_text(text)
  ollama.embeddings(all-minilm, ...)
retrieve(coll, query, k=3)
  chroma top-k nearest
generate_answer(query, chunks)
  ollama.generate(llama3.2:1b, ...)
```

Right now you cannot answer:

- Which chunks were retrieved?
- How many tokens did that answer cost?
- How long did each stage take?
- Did the LLM actually follow the prompt?

That's what step 2 fixes.

STEP 2

Add Langfuse

One import, one decorator per stage. Every trace visible at localhost:3000.

```
@observe()
```

```
@observe(as_type="generation")
```

20 min · the wow moment. Tour the UI together.

STEP 2 · WHAT YOU SEE AT LOCALHOST:3000

Every question becomes a nested trace.

```
ask                                                  2.1s  ·  trace-id 4f2e
  retrieve                                           0.1s  ·  3 chunks
    embed  ·  model=all-minilm                       30ms
  generate_answer  ·  GENERATION                     2.0s  ·  416→42 tok
```

Click any span → see input, output, latency, token usage.

Each span gives you:

- the exact prompt sent
- the exact completion received
- token counts in + out
- latency per stage
- model version used
- any metadata you attach

STEP 3

Add OpenTelemetry + Jaeger

Langfuse + OTel coexist. Langfuse: LLM-native view. OTel: cross-service view.

```
tracer.start_as_current_span("llm.generate")
```

15 min · same pattern, different backend. Today exports to Jaeger at localhost:16686.

STEP 3 · WHY BOTH

They answer different questions.

Langfuse

Answers: "Why did the model say THAT?"

- Prompt + completion inspection
- Token + cost rollups
- Prompt versioning
- LLM-as-judge scoring
- User session grouping

OpenTelemetry

Answers: "Where does this fit in our stack?"

- Cross-service trace propagation
- Today: Jaeger UI · tomorrow: Tempo, Datadog, Honeycomb...
- Vendor-neutral: instrument once
- Metrics + logs alongside traces
- Correlates with your non-LLM code

STEP 4

Break it. Debug it.

I planted a realistic bug. Same answers. Wrong answers. No exception.

```
python step4_debug/rag.py
```

20 min · no peeking at SOLUTION.md — the debugging IS the workshop.

STEP 4 · THE DEBUG FLOW

Work backwards through the trace.

1. **The answer looks wrong.**
   Don't read the code. Read the trace.
2. **Open the prompt sent to the LLM.**
   Is the right context in it?
3. **If no → look at the retrieved chunks.**
   Did we get back the right source?
4. **If no → check the embed spans.**
   What model did each use?
5. **Look for the mismatch.**
   Same code, different signal. That's your bug.

This flow is impossible without per-stage tracing. That's why it's not optional.

TAKE THIS TO PROD

What to change when you're not on a laptop.

Ollama → hosted model	Swap the client; @observe decorators don't change. Langfuse auto-captures OpenAI/Anthropic SDKs.
ConsoleSpanExporter → OTLP	One line. Your spans land in Jaeger, Tempo, Datadog, Honeycomb, your existing APM.
EphemeralClient → persistent vectors	Chroma HttpClient, Pinecone, Weaviate, pgvector. The trace boundary is the same.
Add evaluation loop	Langfuse LLM-as-judge scores every response. Trend the scores. Gate releases on it.
Mask PII before persisting	Langfuse masking hook — strip sensitive fields in prompt/completion before storage.

WHAT YOU LEAVE WITH

Four things to take home.

A working repo

Clone it, extend it, rip it apart. MIT-licensed. Bring to work Monday.

An observability pattern

Applies to any LLM project, any model, any backend. The wrapper is the same.

A real debug reflex

You now know what a retrieval mismatch looks like in traces. You'll spot it in prod.

A production roadmap

Evaluation loops, OTLP export, PII masking — the next 90 minutes at work.

RESOURCES

Where to go next.

This workshop repo	`github.com/gigimcc4/DevOpsDays_llmops`
Langfuse docs	`langfuse.com/docs`
OpenTelemetry Python	`opentelemetry.io/docs/languages/python`
Ollama model library	`ollama.com/library`
ChromaDB docs	`docs.trychroma.com`
Arsinnovate	`arsinnovate.com`

Thank you.

Hire me. Stand up your AI stack · audit your roadmap · subcontract on AI engagements

Questions?

Jeanne McClure, PhD
I: Principal · Consultant · Expert
Ars Innovate Technologies & Consulting

Postdoctoral Fellow,
North Carolina State University

bit.ly/3ONJiqb · jmcclure@arsinnovate.com · linkedin.com/in/jeannemcclure

© 2026 Jeanne McClure, PhD / Arsinnovate. Non-commercial use only. Attribution required.